
Smart Contract Code Review and Security Analysis Report for Cube DEX

Revised 9 September 2026 (statuses updated after the
follow-up review)

Serokell OÜ
31 July 2026



Smart Contract Code Review and Security Analysis Report for Cube DEX

This document may contain confidential information about IT systems and the intellectual property of the Customer, as well as information about potential vulnerabilities and methods of their exploitation. It can be used internally by the Customer, or disclosed publicly after all issues are resolved — at the Customer's decision.

Name	Cube DEX (aka Coffer)
Type	Weighted multi-token AMM; protocol governance; single-sided liquidity
Platform	Solana
Language	Rust (Anchor 0.32.1)
Methods	Architecture Review, Manual Code Review, Functional Testing, Static Analysis, Dependency Audit, Stateful Fuzzing, Proof-of-Concept Exploitation
Commit	<code>070a55dac94fe539c5896feb33ac719c3f6b9d4b</code>
Timeline	2026-06-30 – 2026-07-31
Re-review commit	<code>6f64097c60630676e7fa779b460e8237dd7ac533</code>
Re-review timeline	2026-08-21 – 2026-09-09

Table of contents

1. Introduction
2. Scope
3. Severity definitions
4. Executive summary
5. Checked items
6. System overview
7. Threat model and assumptions
8. Findings
 - High severity
 - Medium severity
 - Low severity
 - Informational
 - Design notes (accepted)
9. Disclaimers



10. Appendix A — Reproduction

11. Appendix B — Tooling

1. Introduction

This report presents the results of a pre-deployment security assessment of the Cube DEX on-chain programs. The review was manual-first, supported by static analysis, dependency scanning, stateful fuzzing, and runnable proof-of-concept exploits executed against a local validator. Two auditors performed the manual line-by-line review independently, cross-checking findings before this report was compiled.

The assessed code is not yet live: mainnet currently runs an earlier version. Every issue in this report is therefore fixable before deployment, without state migration.

2. Scope

Repository: `cubee-contracts` (private)

Commit: `070a55dac94fe539c5896feb33ac719c3f6b9d4b`

Technical documentation: Yes (`docs/ai-context/`)

Tests: Yes (19 TypeScript integration suites, Rust unit tests, 2 Trident fuzz harnesses)

Programs:

Program	Program ID (mainnet)	Lines	Role
<code>cubic-pool</code>	<code>8iQt6j9mcUfFUGaiCpPy89swC3s8YTC8FhVZWfgeZhwu</code>	9,820	Core AMM — pools, swaps, liquidity, per-pool policy
<code>protocol-admin</code>	<code>3jiojHZbjJQ7QLMGSTjFwxVEmx4NtuRy34nLAmsJME81</code>	2,460	Governance — Treasury PDA, privileged routing, upgrades
<code>single-token-liquidity</code>	<code>7BpdUH1tzTSXLuQNo6YpjJ8Eagw8AkrS6cnkxiJdCFS2</code>	1,094	Single-sided deposit (zap)

Out of scope: off-chain operator tooling, the frontend, the SDK, and the backend. Some off-chain tooling (`admin-panel/scripts` , `scripts/cli`) was reviewed when it was necessary for better understanding of smart contracts and some findings were provided, but they should be treated as complementary since this tooling was explicitly out of scope.

3. Severity definitions

Risk level	Description
Critical	Directly exploitable, leads to loss of user funds or full protocol takeover.
High	Significant impact on protocol execution or economics — bypassed access control, defeated safety mechanism, loss of user funds under a reachable configuration, or loss of protocol revenue. Exploitation may require a specific state or role.

Risk level	Description
Medium	Important to fix, but cannot by itself lead to loss of user funds — weakened safety margins, irreversible operator errors, or misleading protocol state.
Low	Correctness and robustness issues with no direct security impact — inconsistent errors, unenforced checks in tooling, missing guards.
Informational	Self-inflicted, frontend-dependent, or observability issues. Not attacker vectors.

4. Executive summary

No Critical issues were found. The AMM’s core safety invariants hold under fuzzing: across ~330k instructions, value conservation and extractable-value bounds held, and every outsider takeover attempt against the Treasury was rejected. However, manual review found a path where an attacker can extract another user’s funds under certain conditions.

The issues that were found cluster in four areas: **access control at protocol bootstrap** (config and Treasury initialization are not gated to the intended authority), **safety mechanisms that can be side-stepped** (the sell-off cap, the surge fee, the mint-extension ban, and the range-manager envelope can each be defeated by an attacker who first shapes the pool state), **governance paths that do not work as documented** (upgrade-authority rotation and program freeze cannot execute at all; deployment verification and the migration runner do not do what they claim), and **deposit-path bugs that miscompute balances or fees** (first-deposit leverage skew, doubled fee subtraction, virtual/actual balance mismatch, and a lost-update race each leave a deposit in an unintended state).

Severity	Count
Critical	0
High	8
Medium	6
Low	13
Informational	7
Design notes (accepted)	2
Total	36

Documentation quality — good. The repository ships a deep-dive architecture document, a mainnet upgrade playbook, and per-instruction rationale. Two documents are stale relative to the code (the `migrate_to_v5` wrapper is documented as a no-op while it mutates state, and the admin-panel README still uses pre-rename instruction names).

Code quality — good. Consistent Anchor idioms, checked arithmetic with `u128` /256-bit widening in the math modules, explicit error enums, and events on state changes. The main defect is the release profile shipping with `overflow-checks = false`, which removes the backstop behind the few unchecked operations.

Architecture quality — good. Clear separation between the AMM, governance, and the zap program; all privileged operations route through a single Treasury PDA signer. The weakness is that this routing is convention rather than enforcement: two initialization instructions in the AMM can be called directly, bypassing the governance wrapper that carries the admin check.

Test coverage — good, with gaps. 19 integration suites cover the lifecycle, Token-2022, ALTs, and the range manager; two stateful fuzz harnesses assert value conservation and pricing monotonicity per transaction. Gaps: multi-token pools (3–10 tokens) are not fuzzed, `single-token-liquidity` is not fuzzed, and one suite passes against an instruction that no operator tool can actually call (`migrate_to_v5`).

Verdict. The code is fit for deployment once the High findings are resolved and the points marked “Client decision” are answered. A few of the issues require a redesign; others are local fixes.

Follow-up review. After fixes, Serokell re-audited the contract at revision

`6f64097c60630676e7fa779b460e8237dd7ac533`, where 29 of the 36 original findings were confirmed resolved and 12 issues were found:

Severity	Count
High	1
Medium	2
Low	4
Informational	5
Total	12

Full details are in the `Cube DEX (aka Coffey) – Follow-up Security Review (Second Round)` report.

5. Checked items

Item	Category	Description	Status	Findings
Signer and caller authorization	Access control	Every privileged instruction is gated on the intended authority.	Failed	H-01, H-02, L-07, L-09
Ownership takeover	Access control	Protocol ownership cannot be seized by an outsider on the assessed in-place upgrade.	Passed	L-05
PDA seeds and bump validation	Solana-specific	PDAs are derived with canonical bumps and pinned by seed constraints.	Passed	—

Item	Category	Description	Status	Findings
Arbitrary CPI	Solana-specific	CPI target programs are pinned; a caller cannot substitute a malicious program.	Failed	I-06, L-10
Account re-initialization	Solana-specific	No <code>init_if_needed</code> ; one-shot accounts cannot be re-initialized.	Passed	—
Duplicate mutable accounts	Solana-specific	The same account cannot be passed twice to double-credit a balance.	Passed	—
Token-2022 extension handling	Solana-specific	Mint extensions that can seize or block vault funds are rejected.	Failed	H-05
Integer overflow and underflow	Arithmetic	All arithmetic is either checked or provably bounded.	Failed	M-01, L-06, L-13
Rounding and precision	Arithmetic	Rounding always favours the pool; no dust extraction path.	Passed	—
Value conservation	Assets integrity	<code>vault = actual_balance + protocol_fees_owed</code> holds after every instruction.	Passed	—
Extractable value	Assets integrity	No sequence of instructions returns more value than it puts in.	Failed	H-08
Deposit and redemption accounting	Assets integrity	Deposits and withdrawals are credited on a consistent basis across instructions.	Failed	H-06, I-01, I-02
Checks-Effects-Interactions	Reentrancy	State is written before external transfers.	Failed	I-02
Slippage protection	Price manipulation	Swaps enforce a caller-supplied minimum output.	Passed	—

Item	Category	Description	Status	Findings
Price manipulation resistance	Price manipulation	Pricing state cannot be shaped cheaply to defeat protections.	Failed	H-03, H-04, H-08, M-02, M-04, M-05, M-06, L-11, I-04
Pause and kill switches	Operational	Pause flags are honoured by every state-mutating instruction.	Failed	H-07, L-01
Upgrade authority management	Operational	Upgrade authority can be rotated, and cannot be lost through a single operator action.	Failed	M-03
Deployment verification	Operational	Post-deploy scripts assert the expected on-chain state.	Failed	L-04, L-12
Event schema stability	Observability	Event layouts are append-only so indexers survive an upgrade.	Failed	I-03, I-05, I-07
Error-code specificity	Observability	Distinct failure conditions raise distinct error codes.	Failed	L-02, L-03
Dependency vulnerabilities	Supply chain	No known advisories in the dependency tree.	Passed	—

6. System overview

Cube DEX is a weighted multi-token AMM in the Balancer V2 family, extended with **virtual balances**: swaps are priced off a per-token `virtual_balance` that the pool owner (or a delegated bot) can move independently of the `actual_balance` actually held in the vaults. This decoupling is the source of the protocol's flexibility and of several findings in this report.

`cubic-pool` holds pools, vaults, and the swap math. A pool carries up to 10 tokens, each with a weight, a virtual balance, an actual balance, an active flag, and an optional per-window sell-off cap. Swap fees split between LPs and the protocol; a **surge fee** may be charged on top when a sell window fills up.

`protocol-admin` owns the Treasury PDA, which is the sole admin signer for the protocol. Every privileged AMM operation is meant to arrive as a CPI from this program, so that the `treasury.admin` check is applied in one place. It also owns program upgrades and the freeze/close path.

single-token-liquidity lets a user deposit one token and receive BPT, sizing the implied basket internally.

Privileged roles

Role	Held by	Can
Treasury admin	Protocol governance key	Every privileged operation: fee rates, pause flags, token registration, fee collection, program upgrades, admin handover.
Supervisor	Optional emergency key	Freeze/unfreeze pools and deactivate tokens. Subset of admin; disabled while unset.
Protocol admin	Treasury PDA	Stored on each pool config; the identity the AMM checks for protocol-level operations.
Pool admin	Pool creator	Per-pool policy: swap fee, sell-off cap, swaps on/off, token activation, range-manager appointment. Renounceable.
Range manager	Optional bot or key appointed by the pool admin	Move virtual balances and weights within a per-update percentage envelope.

7. Threat model and assumptions

Threat model

The adversary is assumed to be an untrusted external party capable of:

- invoking any public instruction,
- supplying arbitrary transaction inputs,
- observing all on-chain state,
- deploying auxiliary programs,
- acquiring significant financial resources, including temporary liquidity through external mechanisms.

The adversary is not assumed to compromise Solana consensus, cryptographic primitives, or trusted administrative infrastructure.

Assumptions

The assessment takes the following as given, rather than as verified findings:

- **External dependencies.** The SPL Token program behaves as documented.
- **Privileged roles.** The treasury admin, protocol admin, pool admin, supervisor, and range manager are trusted parties; their keys are assumed to be securely managed and not compromised. For narrower roles (e.g. supervisor), this trust is bounded by what the protocol actually grants them — a narrower role exercising an admin-only capability remains a protocol issue.

- **Blockchain environment.** Solana consensus is secure, and runtime behaviour matches its documented specification.
- **User behaviour.** Ordinary users may act maliciously and are assumed to control their own private keys.
- **Economic model.** See below.

Economic model assumption

The protocol tracks actual and virtual balances as independent on-chain state, with no formula or code-level invariant linking them; this approach is, to our knowledge, without precedent in comparable multi-token AMMs. Proving the soundness of this design in full is a separate, substantial undertaking outside the scope of this engagement.

The audit therefore assumes the correctness of the protocol's economic model, incentive mechanisms, and underlying mathematical design. It does not assess whether the protocol is economically sustainable under all market conditions, whether its parameters are optimal, or whether the design is free of unintended value-extraction opportunities.

Economic flaws arising from the design itself — incentive imbalances, profitable strategies enabled by intended functionality, market-manipulation opportunities, or adverse market conditions — are considered out of scope, even where related observations are included for informational purposes.

8. Findings

High severity

H-01 — Protocol-fee bypass via unauthenticated config creation

Locations:

- `cubic-pool/src/instructions/admin/initialize_config.rs:39,71`
- `user/initialize_cubic_pool.rs:152`
- `user/swap.rs:187`

Status: Resolved as recommended, with certain nuances

Reproduction: `poc.sh H-01`

`initialize_config` has no caller gate — its only signer is `payer`. The `treasury.admin` check exists solely in the governance wrapper `protocol-admin::pool_initialize_config`, which a direct call bypasses. The protocol fee rate is bounded above but not below, so `0` is accepted, and a pool copies the config rate verbatim.

Anyone can therefore open a pool that charges a swap fee to traders and pays the protocol nothing, since each swap's protocol cut is `fee × 0`.

Zero-fee pools where nobody pays are intended and out of scope. The finding is the rogue-config case: `protocol_fee = 0` with `swap_fee > 0`, where LPs earn and the protocol does not.

Impact: Revenue leak only; no user funds at risk. Recoverable per pool via `pool_set_protocol_fee_rate`.

Client decision: Must the protocol always receive its cut when a pool charges a swap fee? Yes — gate config creation. No — this is by design.

Recommendation: Require the Treasury PDA to sign config creation, or seed `pool.protocol_fee_rate` from a governance constant instead of the config.

H-02 — Missing caller gate masked by a redundant argument

Location: `cubic-pool/src/instructions/admin/initialize_config.rs:30-34,35,61-69`

Status: Resolved as recommended, with certain nuances

Reproduction: `poc.sh H-02` (read-the-code; demonstrated by the H-01 run)

The handler takes a `protocol_admin: Pubkey` argument and requires it to equal the Treasury PDA. That check reads like an admin gate, but the `seeds` constraint already pins the account, so the argument can only ever hold one value: **the check decides what gets stored, not who may call**. There is no caller gate at all.

This is the load-bearing reason H-01 is reachable, and it is easy to miss on review because the code appears to be checking the admin.

Recommendation: Drop the argument, populate `config.protocol_admin` from the constrained account, and gate the instruction on a Treasury PDA signature or restrict it to CPI from `protocol-admin`.

H-03 — First-deposit front-run leaves the pool permanently over-leveraged

Location: `cubic-pool/src/instructions/user/add_liquidity.rs:203-220,300,320-328`

Status: Resolved as recommended, with certain nuances

Reproduction: `poc.sh H-03`

Pool creation and the first deposit are separate transactions, and nothing ties the first depositor to the creator. First-deposit BPT is minted from the pool's **virtual** balances rather than the tokens actually sent, so an attacker who front-runs with dust mints a large BPT position while leaving `actual` at dust and `virtual` large.

The creator's real deposit is then sized on a ratio against that dust, which multiplies every virtual balance and **inflates the pool's leverage permanently**. Because swaps price off `virtual` but are capped by real output, the same swap afterwards drains close to all of the real reserves instead of about half.

Impact: Not theft at mint time — the creator still holds nearly all BPT — but against a permanently over-leveraged pool, so a follow-on swap destroys their liquidity far faster. Cost to attack: a few token units.



Note: beyond the immediate over-leverage, the inflated `virtual_balance` this leaves behind is a landmine for any later math, potentially leading to DoS at certain pool volume: a large enough skew truncates the division at `single-token-liquidity/src/math.rs:72` to zero, tripping `PoolNotSeeded` at `:79`.

Recommendation: Mint first-deposit BPT from the actual deposited amounts; and/or restrict the first deposit to the pool admin; and/or bundle pool creation and the first deposit into one transaction.

H-04 — Sell-off cap is measured against a snapshot the attacker controls

Location: `cubic-pool/src/math/max_selloff.rs:144-148`

Status: Partially resolved as recommended, see L-14

Reproduction: `poc.sh H-04`

The per-window sell cap is resolved against a snapshot of `virtual_balance` taken on the window's **first swap**, not at a wall-clock boundary. `add_liquidity` and `remove_liquidity` grow or shrink `virtual_balance` without touching sell-off accounting.

An attacker therefore deposits to inflate the balance, opens the window with a dust sell so the snapshot locks in the inflated value, and **withdraws the deposit in full before the real sell** — the cap stays inflated by the same factor even though the capital behind it is gone. Two protections fall together:

- **The sell-off brake** passes proportionally more sell volume against reserves the attacker no longer has capital in.
- **The surge fee**, priced off `sold / cap`, reads a large sell as small and collects nothing.

The attacker holds none of the inflation capital at the time of either bypass.

Impact: Not direct theft — the sell still pays price impact — but the safety layer is defeated with no capital held at risk during the drain, and the fee meant to deter it is not collected. Loss lands on the LPs of the drained side.

Recommendation: Keep the sell-off basis and the volumes measured against it in sync: make `add_liquidity` and `remove_liquidity` rescale `selloff_vb_snapshot`, `previous_selloff`, and `current_selloff`, following the virtual balance movement; rescale `previous_selloff` at window rotation.

Additionally, as hygiene, extend today's rotation check beyond swap's sell side — which already runs it — to also fire from `add_liquidity`, `remove_liquidity`, and the swap buy direction, so the window state machine stays current at every site that touches `virtual_balance`.

Notably, rotating the snapshot on a wall-clock boundary still leaves the bypass reachable. Rescaling the snapshot without rescaling `previous_selloff` produces the L-11 lockout instead. Addressing this finding requires a rework of the sell-off accounting and a deliberate design choice by the client.



H-05 — Banned mint extensions are a default, not an invariant

Locations:

- `cubic-pool/src/instructions/user/initialize_cubic_pool.rs:119-120,184`
- `util/token.rs:68-99`

Status: Acknowledged as won't fix — revisited at H-09

Reproduction: `poc.sh H-05`

The effective ban list is `override.unwrap_or(config.banned_extensions)`, so a **pool creator passing `Some(0)` un-bans everything**, and the check runs once at creation.

Un-banning **PermanentDelegate** or **TransferHook** hands the token issuer a backdoor into the AMM's custody: the delegate can empty a vault, and a hook can block withdrawals. The loss lands on the LP who supplied the other, valuable side of the pool.

Impact: In the proof of concept, an override lets a PermanentDelegate token into a pool, the delegate empties the vault (`1e12 → 0`), and the victim's `remove_liquidity` then reverts — funds drained and exit locked. Mitigating factor: the chosen bitmap is stored on the pool and emitted, so LPs and frontends can inspect it before depositing.

Client decision: Should pool creators keep full freedom over the ban list, or is part of it a protocol invariant? Freedom — by design, plus disclosure to LPs. Invariant — apply the mask below.

Recommendation: Apply a non-overridable hard-floor mask for the extensions that reach vault custody:

```
let effective = banned_extensions_override.unwrap_or(config.banned_extensions) |  
CONFIG_HARD_MASK;  
// CONFIG_HARD_MASK = PermanentDelegate | TransferHook (| TransferFee)
```

H-06 — Single-token deposits subtract protocol fees twice

Location: `single-token-liquidity/src/math.rs:137`

Status: Resolved as recommended

Reproduction: `poc.sh H-06`

The invariant is `vault = actual_balance + protocol_fees_owed`, so `actual_balance` already excludes accrued protocol fees, and `add_liquidity` correctly sizes BPT against it directly. The zap instead computes against `actual_balance - protocol_fees_owed`, **subtracting the fee a second time**. Its implied deposit basket no longer matches the pool ratio, so part of the deposit is credited to the pool but earns no BPT.

Impact: The zap depositor is systematically shortchanged and the excess is donated to existing LPs — always short, never over. The gap scales with fees accrued since the last collection: measured at roughly **53% fewer BPT** on a fee-bearing pool (`413,306` versus `873,101`).

Recommendation: Remove the subtraction in `cap_deposit_amounts_to_lp_ratio` and size against `actual_balance`. The two unit tests that encode the current behaviour should be updated with it.

H-07 — Migration instruction silently re-enables every disabled token

Locations:

- `cubic-pool/src/instructions/user/migrate_to_v5.rs:66-93`
- `protocol-admin/src/instructions/pool_migrate_to_v5.rs:1-10`

Status: Resolved as recommended, with certain nuances (residual accepted)

A token can be disabled as a swap input via `set_token_active(index, false)` — the mechanism for cutting off a compromised or depegged mint without pausing the whole pool.

The `migrate_to_v5` handler loops over every token and unconditionally sets `is_active = true`, so a **single migration call re-opens every token that was deliberately disabled**.

The trap is the documentation: the governance wrapper is described in code and IDL as an empty placeholder and a no-op, so an operator running the migration has no reason to expect kill switches to flip.

Impact: Requires pool admin or protocol admin, and a pool admin could re-activate tokens directly anyway. The severity comes from the silent, bulk, mislabelled override during a routine operation.

Recommendation: Remove the reactivation from the handler or make it an explicit opt-in parameter, correct the “no-op” documentation, and add the missing default-key guards.

H-08 — Actual/virtual skew breaks `single-token-liquidity`’s deposit split

Locations:

- `single-token-liquidity/src/math.rs:20-22,45-98` (`compute_allocations`)
- `cubic-pool/src/math/cubic_math.rs:67-77` (`calc_out_given_in`)

Status: Partially resolved via a custom solution — reopened as M-07

Reproduction: `poc.sh` H-08

Once a token’s `actual` balance is inflated relative to its `virtual` balance — M-04 shows one way this arises — this is where that skew turns into direct loss. `compute_allocations` splits a single-token deposit using $W_i = \text{weight}_i * \text{actual}_i / \text{virtual}_i$, a linear stand-in for the real swap curve.

The real curve has a hard ceiling the linear split doesn’t know about — `calc_out_given_in` caps deliverable output at `virtual_balance_out`

($\text{amount_out_calc} = \text{virtual_balance_out} * \text{complement}$, and `complement` is always < 1). Once a token’s `actual` is inflated relative to `virtual`, the split routes a large share of the deposit at that token, and the real swap saturates far below the naive proportional target.

Impact: Silent value loss, not a revert — up to ~99%+ fewer BPT minted than a healthy pool would give for the same deposit, worsening with skew depth and roughly flat across deposit size — captured by the pool’s existing LPs. `minimum_bpt_amount` doesn’t protect the victim: the quote itself is derived from the already-skewed state, so a normal, even tight, slippage margin still clears cleanly.



Recommendation: Since actual/virtual skew can arise by more than one route (M-04 is one example), the fix belongs in `compute_allocations` itself: replace the linear approximation with a split that follows the real swap curve, so the result can't outrun `calc_out_given_in` regardless of how the skew came about. Secondary option: limit how far actual and virtual balances can physically diverge in the first place.

Medium severity

M-01 — Overflow checks disabled in the release profile

Location: `Cargo.toml:8`

Status: Resolved as recommended

Reproduction: `poc.sh M-01` (read-the-code)

The release profile sets `overflow-checks = false`, overriding the Anchor default. Most arithmetic is explicitly guarded with `checked_*` operations and widened to `u128` or 256-bit intermediates, so this is a removed safety net rather than a live exploit — but the unguarded sites (`weighted_math.rs:64,97`, `swap.rs:188`) lose their backstop and would wrap silently instead of aborting the transaction. Enabling checks costs on the order of tens to low hundreds of compute units per swap, under 1%.

Recommendation: Set `overflow-checks = true` in `[profile.release]` before deployment.

M-02 — Range-manager envelope caps velocity, not distance

Location: `cubic-pool/src/instructions/user/range_manager_update.rs:115-124,137-145`

Status: Resolved as recommended, with certain nuances, opened another minor issue, see I-10

Reproduction: `poc.sh M-02`

The pool owner may appoint a range manager that writes virtual balances directly, constrained by a maximum percentage change per update and a minimum interval between updates. The percentage is measured against the live value at each call, with no fixed baseline and no absolute band, so **the limits compound**: 5% per step reaches roughly 2.65× in 20 steps and 131× in 100.

A manager — or a malfunctioning bot — can walk the pool price arbitrarily far using only individually legal updates. In the proof of concept, 50 legal 5% steps move the virtual balance 11.5× and collapse a probe swap's output from `998,002` to `86,935`.

Client decision: Is the range manager a bounded semi-trusted role, or is it trusted like the pool admin? Bounded — the envelope is a trust boundary and must contain distance, not just speed. Trusted — by design.

Recommendation: Measure the percentage against a fixed baseline snapshot, and add absolute per-token bands for virtual balances and weights. See also L-01.



M-03 — Upgrade-authority transfer and program freeze cannot execute

Locations:

- `protocol-admin/src/instructions/transfer_upgrade_authority.rs:19`
- `freeze_pool_program.rs:14`
- `contexts.rs:229-250`

Status: Resolved as recommended

Reproduction: `poc.sh M-03`

Both handlers call the loader helper `set_upgrade_authority`, which takes the **program** address and derives the `ProgramData` address itself. They pass the `ProgramData` address instead, so the CPI is built against `PDA([programdata], loader)` — an address that does not exist and is not among the accounts supplied to `invoke_signed`.

Neither instruction can therefore do anything: every call fails. The runtime rejects the CPI with two errors in sequence:

- `Instruction references an unknown account ...`
- `An account required by the instruction is missing`

The shared `TransferAuthority` context has no account for the program at all, so a client cannot work around it by passing different values.

Impact: No funds at risk, and no attacker involved. The protocol simply has no working way to rotate the upgrade authority away from the Treasury PDA, and no working program freeze. Both are exposed in the IDL and documented in the handlers, so they read as available capabilities; neither is called by any test or operator script, which is why this was never noticed. The upgrade path itself is unaffected: `upgrade_pool_program.rs:15` passes the program address correctly and is the pattern to copy.

A second defect sits behind the first: the transfer uses the unchecked variant, in which the new authority never signs. It is unreachable today because the instruction fails first, but once the address is corrected, a mistyped destination would take the upgrade authority irrecoverably.

Recommendation: Add the program account to the context and constrain `ProgramData` to be its loader PDA; pass the program address to the helper; and switch the transfer to `set_upgrade_authority_checked` so the incoming authority co-signs. The freeze needs only the address fix — it sets the authority to `None`, so there is no one to co-sign, and the unchecked variant is correct there. Cover both with a test, since neither has ever been executed.

M-04 — Lopsided deposits decouple visible depth from priced depth

Locations:

- `cubic-pool/src/instructions/user/add_liquidity.rs:300,309-313,320-328`
- `math/cubic_math.rs:48`

Status: Resolved via a custom solution, opened another issue, see M-08

Reproduction: `poc.sh M-04`

A subsequent deposit credits `actual` with the full amount but grows `virtual` only by the basket-wide minimum ratio. Depositing a large amount of one token and dust of another drives that ratio to near zero, so `actual` jumps while `virtual` barely moves — measured at **+505% real reserves against +0.99% priced reserves**.

Because swaps price off `virtual`, the added real depth is invisible to pricing; because `remove_liquidity` pays off `actual`, the depositor can still redeem it.

Impact: A careful trader is protected — they see the quote and set their own minimum output. Harm lands on anyone judging depth from real reserves instead of the quote: a naive frontend, an arbitrage bot, or an external oracle.

Client decision: Does anything in the ecosystem — frontend, bot, oracle — treat real reserves as the depth or price signal? Yes — this is exploitable deception. No — it is a shallow pool by design.

Recommendation: Refund the excess as dust after minting, as the zap does, or revert on a disproportionate deposit. Shares a root cause with I-01 and I-02.

M-05 — Splitting a swap avoids the surge fee

Locations:

- `cubic-pool/src/math/surge_fee.rs:56-91`
- `user/swap.rs:207-231`

Status: Resolved via a custom solution, opened another minor issue, see I-12

Reproduction: `poc.sh M-05`

The surge fee exists to penalise large sell-offs and rises with how full the sell window is. But the rate is sampled at the swap's **endpoint** fill and then applied to the entire output, instead of integrating the rate across the range the swap actually traverses. Since the curve rises, splitting one sell lets most of the volume settle at a low-fill rate.

With a cap of 100 and a curve charging 0% up to 80% fill and 30% at full: one swap of 100 pays 30. The same 100 split as 80 + 20 pays 0 on the first chunk and 6 on the second — **6 in total, five times less**.

Cutting the charged part finer lowers it again: 80 + 10 + 10 pays 0 + 1 + 3 = **4**. Cutting the untaxed part gains nothing — five equal chunks of 20 still pay 6, because the last chunk still ends at full fill.

Impact: The brake is defeated by anyone willing to send several transactions. Loss of deterrence and of fee revenue, not direct theft.

Recommendation: Charge the integral of the rate from the pre-swap fill to the post-swap fill, which makes the fee path-independent.

M-06 — `range_manager_update`’s “set” semantics create a lost-update race with concurrent ops

Locations:

- `cubic-pool/src/instructions/user/range_manager_update.rs:113-124` (percentage bound, checked against the live value)
- `cubic-pool/src/instructions/user/range_manager_update.rs:148-151` (unconditional overwrite)

Status: Resolved as recommended

The range manager computes its intended `new_value` off-chain against whatever `virtual_balance` it last observed, then submits a transaction that unconditionally sets `virtual_balance[i] = new_value`, not accounting for balance changes that happened in between.

The intention, apparently, is to change virtual balance relative to the current value, since setting an absolute value that disregards the current contract state rarely makes sense — meaning of a `range_manager_update` call that sets balance to 1001 changes depending on whether current balance is 1000 or 10000. The current implementation does not reflect this intention — it always overwrites unconditionally.

Impact: If any swap or liquidity operation happened while the virtual-balance update was landing, its effect on the price is reset — exploitable by anyone who can time a swap into it. This stems from Solana’s read-then-submit latency, not an operator mistake, so the range manager cannot simply avoid it by being careful.

Recommendation: Avoid off-chain calculations that depend on the contract’s state, since that state can change by the time the transaction lands. If the intention is to make the contract approach prices dictated by an external oracle, pass those target prices to a dedicated instruction and perform the further computation on-chain, this way providing atomicity.

Depending on the use case, simpler options may be available. For instance, the virtual balance update could use a compare-and-swap idiom if no high concurrent activity is expected on the pool.

Low severity

L-01 — Range-manager updates ignore the pool kill switches

Location: `cubic-pool/src/instructions/user/range_manager_update.rs:20-30`

Status: Resolved as recommended

Swaps and liquidity operations check `pool_enabled` and `swaps_enabled`; `range_manager_update` checks only `range_manager_enabled`. Pausing a pool therefore does not stop the range manager from rewriting virtual balances and weights.



Since the manager can only be disabled by the pool admin, governance has no lever to contain a misbehaving one — which is what makes M-02 unbounded in practice.

Client decision: Should a protocol-level pause also stop the range manager? Yes — it's a kill switch and should be total. No — the range manager is out of governance's scope by design.

Recommendation: Require `pool_enabled` in the instruction, and give the protocol admin an override to force `range_manager_enabled = false`.

L-02 — One error code for three unrelated failures

Location: `cubic-pool/src/instructions/user/add_liquidity.rs:120-140,291`

Status: Resolved as recommended

`add_liquidity` raises `InvalidAmounts` for an array-length mismatch, an all-zero first deposit, and a deposit that would activate a sidelined token or sideline a live one. All three revert correctly, but a caller cannot distinguish a client-side mistake from a pool-state constraint.

Recommendation: Split into distinct codes, e.g. `InvalidArrayLength`, `FirstDepositRequiresNonzero`, `TokenLivenessMismatch`.

L-03 — Constraint order makes an error branch unreachable

Locations:

- `cubic-pool/src/instructions/user/set_max_selloff.rs:38-39`
- `set_swap_fee_rate.rs:13-14`

Status: Resolved as recommended

Both instructions check `authority == pool.pool_admin` before checking that the pool admin is not the default key. On a pool whose admin has been renounced, the first check requires signing as the all-zeros key, which is impossible, so it always returns `Unauthorized` and the `PoolAdminDisabled` branch is dead. Sibling instructions order these the other way round.

Impact: No security gap — an inconsistent error code only.

Recommendation: Swap the two constraint lines so the default-key check runs first.

L-04 — Deployment verification never checks the admin it promises to check

Location: `admin-panel/scripts/03-verify-deployment.sh:8,13,66-77`

Status: Resolved as recommended

The script's header states that it verifies the admin matches the expected value and documents an `EXPECTED_ADMIN` variable, but that variable appears only in comments — no line reads it. For the Treasury it dumps the raw account and instructs the operator to compare by eye. It never decodes `Treasury.admin`, never compares, and exits successfully regardless.

Impact: A wrong or hijacked admin passes deployment verification silently. This is the residual risk behind L-05.

Recommendation: Decode the Treasury account in a helper and fail with a non-zero exit when `admin  $\neq$  EXPECTED_ADMIN`.

L-05 — Treasury initialization is unauthenticated (front-run window)

Location: `protocol-admin/src/instructions/initialize.rs:9-46`

Status: Resolved as recommended, with certain nuances

`initialize` has no caller gate — the only signer is `payer` and the admin is a free argument. The Treasury is a one-shot PDA, so **whoever lands the transaction first owns it permanently**: the protocol admin for every pool and the sole signer for the entire governance surface. Between program deployment and the legitimate initialization, anyone watching can seize it. The instruction also accepts the default key as admin.

The window is closed on this deployment, but not by the instruction. `MAINNET_UPGRADE_PLAYBOOK.md` Phase 3 upgrades `cubic_pool` in place inside the existing ProgramData PDA under the same program ID, and Phase 0 asserts against the Treasury already live on mainnet. `protocol_admin` is upgraded in place as well. Nothing calls `initialize` during the rollout, and it would revert if it did, since the PDA is one-shot and already taken.

Two paths still open it: rehearsals on devnet and testnet, which deploy under fresh program IDs, and any disaster-recovery redeploy under new IDs. Nothing detects a hijack in either case — `00-init-treasury.ts:52-58` exits 0 when a Treasury already exists, so a front-run reads as a clean deploy, and the verification script never decodes `Treasury.admin` (L-04).

Impact: No exposure on the assessed in-place upgrade. On any deployment that is not an in-place upgrade, an outsider takes permanent ownership of the entire governance surface for the cost of account rent, and the operator tooling reports success.

Recommendation: Gate `initialize` on an expected bootstrap authority, reject the default key as admin, and fix L-04.

L-06 — Unchecked decimal-normalization overflow in the invariant calculation

Location: `cubic-pool/src/math/weighted_math.rs:61-69`

Status: Resolved as recommended

`WeightedMath::calculate_invariant`'s decimal-normalization branch for `dec  $\geq$  6` divides by `10u128.pow(dec - 6)` with no overflow check, unlike its sibling `checked_mul` branch for `dec < 6`. With the release profile's `overflow-checks = false` (M-01), the exponentiation wraps at `dec  $\geq$  45`. An `InvalidTokenDecimals` error variant (`errors.rs:81`) exists for exactly this case but is never constructed or checked anywhere, so the intended cap on mint decimals was never wired in.



Reachable only on a pool's first deposit, by the pool creator, using a mint they would have to deliberately create with `decimals ≥ 45` — outside any real token's range, but not outside what a malicious pool creator could mint themselves.

Impact: Miscalculation in processing the first deposit. No real token has decimals anywhere near this range, so nothing is exposed today; kept Low because the guard clearly meant to prevent it (`InvalidTokenDecimals`) is dead code, and the cap it should enforce is currently unenforced anywhere else.

Recommendation: Verify token decimals is below cap in

`cubic-pool/src/instructions/user/initialize_cubic_pool.rs:187:`

```
let decimals = parse_mint_decimals(mint_account)?;  
require!(decimals ≤ 18, InvalidTokenDecimals)
```

The exact cap is a product choice.

L-07 — `withdraw_sol` never checks that `source` belongs to the config it authenticates against

Location: `cubic-pool/src/instructions/admin/withdraw_sol.rs`

Status: Resolved as recommended

`source` is only checked for program ownership (:59), not that it belongs to `config`, like sibling instructions do (e.g. `cubic-pool/src/instructions/user/set_swaps_enabled.rs:16`). Any `protocol_admin` can drain any pool/config account, not just its own.

Impact: Latent — today one `protocol_admin` (the Treasury PDA) covers everything. Becomes exploitable once `protocol_admin` diverges from the Treasury PDA (see L-09).

Recommendation: Add the `pool.config = config.key()` check the sibling instructions already use.

L-08 — `single-token-liquidity`'s SOL-withdrawal guard can never be satisfied

Location: `single-token-liquidity/src/instructions/admin/withdraw_sol.rs`

Status: Resolved as recommended

The instruction's documentation promises the ability to drain lamports from any account the program owns (:35-37); the guard (:44-47) checks `source.owner == &crate::ID`. No account that could satisfy the guard exists anywhere in the codebase.

The program's only associated account, the "helper" PDA (`single-token-liquidity/src/instructions/user/deposit_single_token.rs:40`), is never created or assigned by any instruction — it exists solely as a CPI-signing authority, so it remains owned by the System Program indefinitely.

Impact: The documented recovery path does not work as described, and any SOL sent — deliberately or by mistake — to a `helper` PDA (a legitimate, publicly derivable, pool-associated address) is permanently unrecoverable through any sanctioned instruction.

Client decision: Is recovering SOL stuck at the `helper` PDA worth building for? Yes — add the signed-transfer path below. No — correct the documentation to stop promising a recovery path that does not exist.

Recommendation: Add a signed `system_instruction::transfer` CPI path — the helper only needs to sign, not own, to move its own lamports — taking the pool account to derive and verify the seeds.

L-09 — Protocol-admin rotation never re-checks that the new admin is the Treasury PDA

Location: `cubic-pool/src/instructions/admin/initiate_protocol_admin_transfer.rs`

Status: Acknowledged as won't fix

`initialize_config` enforces `protocol_admin = Treasury PDA` at creation (`initialize_config.rs:66-69`), but that check exists only there.

`initiate_protocol_admin_transfer` accepts an arbitrary `new_admin` argument with no PDA check, so a config's `protocol_admin` can be rotated away from the Treasury PDA entirely — the invariant this report relies on elsewhere (H-01, L-07).

Notably, `admin-panel/PROTOCOL_ADMIN_ROTATION_RUNBOOK.md` suggests that

`initiate_protocol_admin_transfer` serves primarily for fixing configs initialized with incorrect admin; since the incorrect initialization incident, `initialize_config` has been added the `protocol_admin = Treasury PDA` check. So behaviour of `initiate_protocol_admin_transfer` should be similar to one of `initialize_config` nowadays.

Impact: The new key then directly controls every protocol-admin-gated instruction on every pool under that config, bypassing the Treasury's governance routing. The Treasury's freeze/unfreeze circuit breaker also becomes permanently inert for that config's pools, since it signs its CPI as the Treasury PDA, which no longer matches.

Recommendation: Require the incoming `new_admin` to equal the Treasury PDA, or otherwise restrict the destination the same way `initialize_config` does.

L-10 — `debug_withdraw_liquidity` performs an unchecked CPI with a caller-controlled program, vault, and recipient

Location: `cubic-pool/src/instructions/admin/debug_withdraw_liquidity.rs:100-148`

Status: Resolved as recommended

Every check this instruction needs — an amount bound, a `token_program` match, vault derivation, recipient validation — is present in the file, directly above the CPI, but commented out.



`token_program_account`, `vault_account`, and `recipient_account` are all read unchecked from `remaining_accounts`, then used in a `Transfer` CPI signed with the pool PDA's own seeds, forwarding `remaining_accounts` unmodified.

Reachable only by the protocol admin (via the Treasury) on an already-disabled pool — the instruction's documented "brick the pool" purpose — so exploiting it requires an actor who could already legitimately drain the pool's real vaults through the sanctioned path.

Impact: Within that gated scenario, the missing checks turn a scoped capability — drain this pool's own vaults — into an unscoped one: a CPI into any attacker-supplied program, with the pool PDA as a valid signer, moving an unbounded amount between arbitrary accounts. This widens the blast radius of a compromised or malicious protocol-admin key beyond every sibling instruction (`collect_protocol_fees` keeps the equivalent checks live).

Recommendation: Restore the existing validation blocks. At minimum, keeping the `token_program_account` equality check alone removes the arbitrary-program-CPI vector.

L-11 — Deflating virtual balance at a window rotation locks out honest sells

Location: `cubic-pool/src/math/max_selloff.rs`, `check_and_advance` function

Status: Resolved as recommended, opened another minor issue, see L-14

Reproduction: `poc.sh` L-11

The mirror image of H-04. `previous_selloff` (the prior window's decay-weighted sell volume) carries into the next window without being rescaled to the freshly-taken snapshot. If `virtual_balance` shrinks right as a window rotates — an LP's `remove_liquidity`, or an ordinary buy — the new cap can land **below** the untouched carryover alone: every sell of that token then reverts with `MaxSelloffExceeded`, with zero real sell pressure since the rotation, until enough of the window decays away.

Impact: Not fund loss — nothing moves — and the lockout is bounded ($\sim \text{period} * (1 - \text{cap_new}/\text{previous_selloff})$) and self-clearing with no operator action.

Recommendation: Same as in H-04.

L-12 — No working operator tool for the migration instruction

Locations:

- `scripts/cli/migrate-and-init-alt.ts:183,222`
- `admin-panel/README.md:75`

Status: Resolved as recommended

After the `migrate_pool_v4` → `migrate_to_v5` rename the instruction ships correctly and the test suite calls it, but the only operator runner still calls the renamed-away methods. Those names are no longer in the IDL, so Anchor throws at resolution time and the script cannot send the transaction. The admin-panel README documents the old names as well.

Impact: No on-chain defect — a tooling gap, and one that fails loudly: the runner throws on the first invocation rather than sending a wrong transaction. The danger is false assurance — a green test suite conceals that the production runner is dead, and this is discovered during a migration window rather than before it.

Recommendation: Update the runner and the README to the current instruction names, and cover the migration path in an operations smoke test.

L-13 — Single-token deposits revert once a pool's balance grows past a fixed-point ceiling

Location: `single-token-liquidity/src/math.rs:65-68`

Status: Resolved as recommended

`compute_allocations` sizes each token's share of a single-token deposit as `weight_bps * actual * FP_ONE / (virtual * WEIGHT_SCALE)`, applying the `1e18` fixed-point scale before dividing. The intermediate is `u128`, so the product overflows once `weight_bps * actual` passes `u128::MAX / 1e18`, roughly `3.4e20`. For a token held at 50% weight that is an actual balance of `6.8e16` base units, about 68M tokens at 9 decimals. The `checked_mul` catches it and the deposit returns `MathOverflow`.

The ceiling is read off the pool's balance rather than the deposit, so the failure is not per-transaction: once a pool crosses it, every single-token deposit into that pool reverts, a one-unit deposit included, and the pool does not drift back below the line on its own. High-supply mints reach the threshold in ordinary operation. The same shape recurs at `:84` and in the deposit-capping helper (`:152-155`, `:171-175`), at far higher thresholds.

Impact: No funds at risk — the transaction reverts, the core pool is unaffected, and liquidity can still be added through the multi-token path. Single-token deposits become permanently unavailable for a whole class of pools, which is a feature outage rather than an exposure.

Recommendation: Divide before applying the fixed-point scale so the intermediate stays bounded, and reorder the capping step the same way. Alternatively route both through a 256-bit intermediate, as `cubic-pool` already does for its own fixed-point math (`math/log_exp_math.rs:247`, `mul_div_down`).

Informational

Self-inflicted, frontend-dependent, or observability issues. Not attacker vectors.

I-01 — Over-supplied deposit amounts are donated with no refund

Location: `cubic-pool/src/instructions/user/add_liquidity.rs` (subsequent-deposit branch)

Status: Resolved via a custom solution

BPT is minted on the minimum ratio across the basket, but the full amount of every token is credited to the pool. Anything above that ratio is donated to existing LPs with no BPT and no refund, while `remove_liquidity` does refund unburned BPT — the two directions are asymmetric. Only bites on a frontend miscalculation or a ratio shift between quote and execution.



Recommendation: Refund the excess as dust after minting, as the zap does, or revert on a disproportionate deposit.

I-02 — No minimum-output guard on subsequent deposits

Location: `cubic-pool/src/instructions/user/add_liquidity.rs:223` (guard present) versus the subsequent branch

Status: Resolved as recommended, with certain nuances

The first-deposit path requires `bpt_amount > 0`; the subsequent path does not. The minimum ratio floors to zero when any leg is tiny, and transfers occur before the mint, so with

`minimum_bpt_amount = 0` an undersized leg causes the entire basket to be donated for zero BPT.

Extreme case of I-01, and the one place where the Checks-Effects-Interactions ordering has a visible consequence.

Recommendation: Add `require!(bpt_amount > 0, InsufficientBptOut)` to the subsequent path.

I-03 — Swap event breaks the append-only field convention

Location: `cubic-pool/src/events.rs` (Swap)

Status: Resolved as recommended

Borsh encodes positionally and the repository convention is to append new fields after `timestamp`.

`surge_fee_amount` was inserted before it, shifting the offset of every following field: an indexer decoding an older event with the new IDL misreads it. Logs only, and not yet deployed, so it is free to fix now.

Recommendation: Move `surge_fee_amount` after `timestamp` before deployment.

I-04 — Sell-off cap and surge fee are inactive by default

Locations:

- `cubic-pool/src/instructions/user/initialize_cubic_pool.rs:165`
- `user/swap.rs:154-168,207`

Status: Acknowledged as won't fix

A new pool initialises every token slot with `max_selloff_pct = 0`, which the swap path treats as “no limit”, and the surge fee only applies inside an active window. Both the sell-side rate limit and the surge fee therefore stay inert until a pool admin opts in. This is by design, but it means the protections do not exist unless configured — and it is why H-04 and M-05 have no live impact today.

Client decision: Will the sell-off cap and surge fee be enabled on production pools? If they stay off, H-04 and M-05 have no live impact — but no dump protection exists either.

Recommendation: Treat `set_max_selloff` as a required deployment step for pools that need dump protection, and consider a safe non-zero default.

I-05 — Freeze events omit the previous state

Location: `protocol-admin/src/events.rs:227-241`

Status: Resolved as recommended

`PoolsFrozen` and `PoolsUnfrozen` carry only the treasury, authority, pool count, and timestamp — no previous flag value and no per-pool keys. An indexer cannot distinguish a real transition from an idempotent re-freeze, nor tell which pools were affected. Other events in the same program follow the old/new convention.

Recommendation: Add the per-pool previous state and the pool keys to both events.

I-06 — Treasury-signed arbitrary CPI (present on mainnet, fixed in `main`)

Location: `protocol-admin/src/instructions/pool_freeze.rs:57` (the fix, commit `b05a943`)

Status: Was resolved at the moment of initial review, belonged to V4

`freeze_pools` and `unfreeze_pools` perform a CPI signed by the Treasury PDA. In the version currently deployed on mainnet the target program was taken from a caller-supplied account with no constraint, so a caller could substitute a malicious program and have Anchor invoke it with the Treasury signature and the pool and vault accounts attached — a direct vault-drain path.

In the reviewed code the target is pinned with `#[account(address = cubic_pool::ID)]`. Recorded here for traceability only.

Client decision: Is the version currently on mainnet being replaced by this deployment? If not, **this is Critical for the live version.**

Recommendation: No action in `main`. If the live version is not replaced by this deployment, ship the same pinning to it before anything else.

I-07 — Single-token-deposit event sums raw amounts across mints of different decimals

Location: `single-token-liquidity/src/instructions/user/deposit_single_token.rs:385-403`

Status: Resolved as recommended

The emitted event's `dust_refunded` field sums refund amounts across mints that can have different decimals, so the total is not a meaningful number. This breaks the convention followed by the event's sibling fields (`allocations`, `deposited_amounts`), which are index-aligned `Vec<u64>`.

Recommendation: Change `dust_refunded` to an index-aligned `Vec<u64>`, matching the sibling fields. This is a breaking change to the event's wire format; update indexers and the SDK in lockstep.

Design notes (accepted)

Inherent properties of the design rather than defects. No fix proposed; listed so the Customer can confirm each is understood and accepted.

S-01 — Just-in-time liquidity can skim swap fees

Status: Acknowledged — accepted by design

Depositing immediately before a large swap and withdrawing immediately after captures a pro-rata share of that swap's LP fee. The bar is high here: a weighted multi-token pool requires all tokens proportionally, only the LP portion is skimmable (protocol and surge fees go to the protocol), and Solana has no public mempool to observe the target swap.

S-02 — Withdrawing liquidity amplifies price impact

Status: Acknowledged — accepted by design

`remove_liquidity` shrinks virtual and actual balances by the same ratio, and since price is set by the virtual balance, pulling liquidity deepens the impact of the next swap — the same lever as M-02, reached through a legitimate LP exit. Leverage is bounded by the exiting LP's own share, value conservation holds, and counterparties remain protected by the slippage gate.

9. Disclaimers

Assessment disclaimer. The programs given for audit have been analysed according to current industry best practice at the date of this report, for cybersecurity vulnerabilities and issues in the source code, its compilation, deployment, and intended functionality. This report makes no statement or warranty on the security of the code and cannot be considered a sufficient assessment of its utility, safety, or bug-free status. We recommend proceeding with several independent audits and a public bug bounty programme.

Technical disclaimer. Programs are deployed and executed on a blockchain platform. The platform, its runtime, the compiler toolchain, and other software related to the programs may themselves contain vulnerabilities that lead to exploits. This audit cannot guarantee the explicit security of the audited programs.

Scope disclaimer. This assessment covers the three on-chain programs at commit `070a55dac94fe539c5896feb33ac719c3f6b9d4b`. Off-chain components are out of scope. Findings marked conditional depend on a configuration that is not currently active in production.

Appendix A — Reproduction

`bash scripts/audit-proofs/poc.sh <id>` runs one proof, or all of them without an argument. On-chain proofs need a running `solana-test-validator`; setup is in `README.audit.md`.

ID	Finding	Type	Observed result
H-01	Protocol-fee bypass	on-chain	rogue pool collects 0 protocol fees versus 6,000,000 on the control pool
H-02	Missing caller gate	read-only	folded into H-01

ID	Finding	Type	Observed result
H-03	First-deposit front-run	on-chain	victim pool leverage orders of magnitude higher; the same swap drains far more
H-04	Sell-off snapshot outlives withdrawal	on-chain	deposit inflates cap 3×, is withdrawn in full, then the stale cap covers a 2×-honest-cap sell at zero capital risk, with zero surge fee
H-05	Banned-extension override	on-chain	PermanentDelegate token admitted; vault emptied <code>1e12 → 0</code> ; victim exit reverts
H-06	Double-subtracted protocol fees	on-chain	same deposit mints <code>413,306 BPT</code> versus <code>873,101</code> after collection (−53%)
H-08	Skewed split in single-token deposit	on-chain	one seed deposit → 500.5× actual/virtual skew; the real <code>deposit_single_token</code> mints <code>39 BPT</code> versus <code>221,505</code> for the same deposit into an equally-sized unskewed pool (0.02% of fair)
M-01	Overflow checks disabled	read-only	<code>Cargo.toml:8</code> sets <code>overflow-checks = false</code> , overriding the Anchor default; the unguarded sites lose their abort-on-wrap backstop
M-02	Range-manager envelope	on-chain	50 legal 5% steps move the virtual balance 11.5×; probe swap <code>998,002 → 86,935</code>
M-03	Authority transfer cannot execute	on-chain	CPI targets a non-existent address; <code>An account required by the ;authority instruction is missing</code> unchanged
M-04	Hidden-liquidity skew	on-chain	real reserves +505% while priced reserves +0.99%; hidden depth redeemable
M-05	Split-swap surge dodge	cargo test	one swap pays 30; split 80+20 pays 6; split 80+10+10 pays 4
L-11	vb-deflation sell-lockout	on-chain	LP deflation at rotation locks out an honest dust sell that a no-deflation control pool accepts; recovers on its own after the predicted decay window

Appendix B — Tooling

Four scanners and a stateful fuzzer were run over the full scope. **No finding in this report originally came from them.** The scanners look for template-level Solana mistakes — missing signer or owner checks, `init_if_needed`, shared PDAs — and none of those patterns occur in this codebase, every hit on this front was a false positive. The fuzzer found no violation at all. Sec3 X-Ray found two false positives, plus seven arithmetic overflow issues that are already covered by this report.

Analysing `cubic-pool` with Sec3 X-Ray required a straightforward rewrite due to known limitations of the tool.

Tool	Configuration	Result
<code>cargo-audit</code>	RustSec database against <code>Cargo.lock</code>	0 vulnerabilities across 351 crates
<code>cargo-clippy</code>	<code>arithmetic_side_effects</code> , <code>indexing_slicing</code> , all 3 programs	1 lead, remainder known or noise
Radar	Auditware, 34 templates × 96 files	3 High — all false positives
Sec3 X-Ray	v0.0.6, LLVM-IR static analysis	2 false positives; 7 findings related to the use of unchecked arithmetic, already covered above
Trident	Ackee 0.12.0, ~330k instructions over 1000 iterations, 2 harnesses	No invariant violated, no panic

Audit workflow also drew on Claude Code skills for context-building, entry-point mapping, and finding verification: [Trail of Bits skills](#) (CC-BY-SA-4.0), [secskills](#) (MIT), and [solana-dev-skill](#) (MIT).

The one lead worth acting on: nine call sites use `anchor_spl::token_interface::transfer` (six in `cubic-pool`, three in `single-token-liquidity`), which wraps the deprecated unchecked SPL `Transfer` instruction rather than `transfer_checked`. It intersects H-05 — an un-banned `TransferFee` mint reverts on transfer and locks the pool.

Switching to `transfer_checked` alone does not close this: the vault would then receive `amount - fee` while `add_liquidity` credits the full amount (`add_liquidity.rs:310-312`, no re-read of the vault balance), replacing the revert with a silent break of the `vault = actual_balance + protocol_fees_owed` invariant that Checked items marks as Passed. Fee-bearing mints need the hard-floor mask from H-05, or balance-delta accounting, on top of `transfer_checked`.

Fuzzed invariants, asserted after every transaction: value conservation, positive virtual balances, weights summing to 10000, a BPT supply floor, no extractable value, a non-decreasing swap value function, monotone fee accrual, and a Treasury admin that is always one of the two legitimate keys — **16,675 outsider takeover attempts, all rejected**. Not fuzzed: `single-token-liquidity`, pools of 3–10 tokens, and Token-2022 mints with extensions (covered by H-05’s proof of concept instead).